Name: Cecilia Martinez Student ID: 112550071

Assignment 1: Analysis of SVD for Fine-Tuning LLMs

GitHub: https://github.com/cecilia-martinez-cs12/BigData2026-Asg1_112550071

1. Question 1: Accuracy vs. target rank

Accuracy	Original LLM	Fine-tuned (r = 16)	Fine-tuned (r = 64)	Fine-tuned (r = 256)
GSM8K	3.5%	15.7%	20.4%	27.0%
MATH	0.1%	3.7%	4.7%	5.4%
Trainable params	0	11.3M (0.9%)	45.1M (3.6%)	180.4M (14.6%)
Final training loss	-	0.455	0.414	0.375

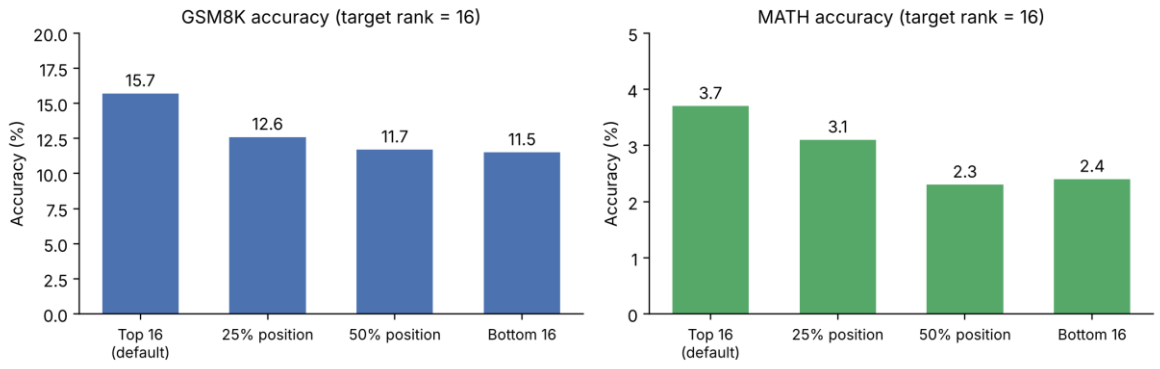
- **The original model almost never answers correctly** (3.5% / 0.1%). Llama-3.2-1B since is a base model that is not instruction-tuned, it often continues the prompt instead of solving the problem and doesn't write a clear final answer. 956 of 1,000 MATH outputs couldn't be parsed. After fine-tuning, GSM8K unparsed outputs drop to 0, so part of the gain comes from learning the answer format and part from better reasoning.
- **Accuracy increases monotonically with rank on both datasets** (GSM8K 15.7 → 20.4 → 27.0%; MATH 3.7 → 4.7 → 5.4%). A larger rank gives more trainable parameters and lets the update cover more principal singular directions of every weight matrix, so the model can learn a more expressive update. The lower final training loss (0.455 → 0.375) confirms the extra capacity is used. With only about 195 optimizer steps (24,975 / 128), capacity is still the bottleneck, so accuracy has not saturated at r = 256.
- GSM8K problems are short arithmetic word problems with integer answers. MATH problems are competition-level, need longer derivations and LaTeX answers, and 1B parameters with 256 output tokens are often not enough: 457–545 MATH outputs remain unparsed or without a final answer. MATH accuracy therefore stays low, and the absolute gain per rank step is small.

2. Question 2: Training time vs. target rank

	Fine-tuned (r = 16)	Fine-tuned (r = 64)	Fine-tuned (r = 256)
Training Time (min.)	63.6	66.2	91.1

- **Training time grows with rank, but much less than linearly.** Rank ×4 from 16 to 64 adds only about 4% time, while 64 → 256 adds about 38% (16 → 256 is +43% for 16× more trainable parameters).
- **Most of the cost is the frozen backbone.** Every step must run the forward and backward pass through all 1.24B frozen weights, regardless of rank. The adapter adds only r·(m+n) multiply-adds per token for an m×n layer, compared with m·n for the base weight. For a 2048×2048 projection that is about 1.6% extra at r = 16 and 6% at r = 64, which is barely visible, but 25% at r = 256.
- **Why 256 becomes slower:** in this implementation the adapter branch ($x \cdot B^T \cdot A^T$) runs in FP32 (autocast is disabled in PiSSALinear), which is much slower on a T4 than the FP16 tensor-core path of the base weights. AdamW also has to update and store states for 180M FP32 parameters (more memory traffic than r = 64), and DataParallel must synchronise more gradients. In SVD initialization is the same for all ranks and is not included. Timings come from single runs on shared Kaggle GPUs; rank-16 runs ranged from 63.6 to 68.3 min.

3. Question 3: Choosing different singular-value ranges (target rank = 16)



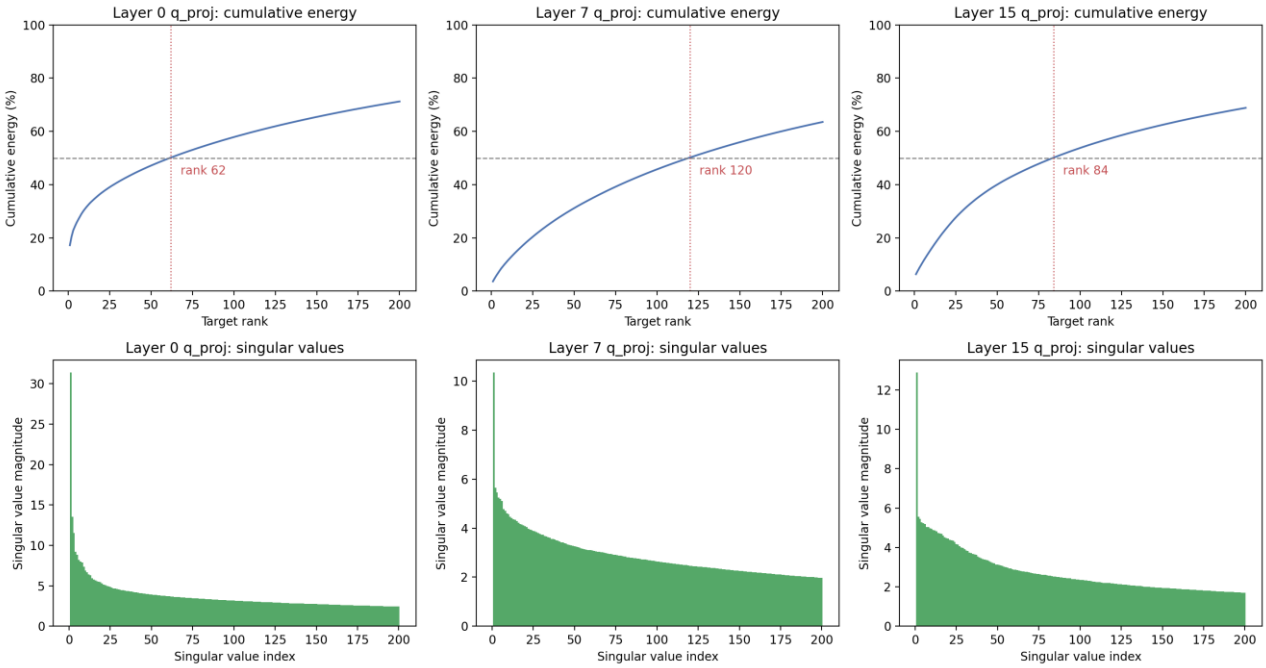
- **The top-16 principal components perform best on both datasets** (15.7% / 3.7%). Accuracy decreases as the window moves toward smaller singular values: GSM8K 15.7 → 12.6 → 11.7 → 11.5%, MATH 3.7 → 3.1 → 2.3 → 2.4%. The final training loss rises in the same order (0.455 → 0.495 → 0.496 → 0.507).

Name: Cecilia Martinez Student ID: 112550071

- **Why:** During training the weight change is $\Delta W \approx \Delta A \cdot B + A \cdot \Delta B$: every step of A is multiplied by B vice versa. With the top components ($\sqrt{\sigma_1} \approx 5.6$ for layer 0) each optimizer step changes the weight noticeably. With the bottom components ($\sigma \leq$ 1e-4 for q_proj in Q4, $\sqrt{\sigma} \approx 0.01$) A and B start almost at zero, so the same steps barely change W. With only about 195 updates at lr 2e-5, the top adapter therefore adapts much more. In addition, $Wv_i = \sigma_i u_i$: the pretrained layer strongly uses the input directions of the top components and almost ignores those of the bottom ones, so tuning the top subspace modifies the part of the computation that actually drives the output
- **All variants still beat the original model,** since even a weak adapter learns the answer format. With 1,000 test samples the standard error is about ±1.0 point for GSM8K and ±0.5 point for MATH, so the differences among the 25%, 50% and bottom settings are within noise. The clear conclusion is top vs. the rest, and it holds on both datasets.

4. Question 4: SVD energy analysis of q_proj (original model, layers 0, 7, 15)

	Layer 0	Layer 7	Layer 15
Minimum target rank for q_proj (≥ 50% energy)	62	120	84
Share of full rank (2048)	3.0%	5.9%	4.1%
Largest singular value σ_1	31.36	10.34	12.88



- **Energy is concentrated in a few directions, but the tail is long**. Half of the energy is reached with only 3–6% of the 2048 directions (62, 120 and 84). After that the curves rise slowly: the first 200 singular values hold only about 71%, 63% and 69% of the energy for layers 0, 7 and 15. At the other end, the smallest singular values are practically zero (≤ 1e-4), so a few input directions are almost ignored by q_proj.
- **Layer 0 is the most concentrated** (minimum rank 62). Its largest singular value ($\sigma_1 = 31.4$) alone holds about 17% of the energy. A possible reason is that the first layer works on the token embeddings, which are dominated by a few strong directions. Layer 7 has the flattest spectrum ($\sigma_1 = 10.3$, first value about 3–4% of energy, rank 120), suggesting that middle layers spread their computation over many directions. Layer 15 lies in between (rank 84).
- **Link to Q1/Q3:** a rank-16 subspace holds only a small part of q_proj's energy (roughly 15–35% depending on the layer, from Figure 2), which is consistent with accuracy still rising up to r = 256 in Q1. The bottom singular values are near zero, which is consistent with the bottom-16 run having the highest training loss in Q3. Note that the 50%-position run did about as poorly, so leaving the top directions matters more than how small σ is.